



**POLITECNICO
MILANO 1863**

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

SmartDesk

Piattaforma per la Gestione di Spazi Coworking

Documentazione Tecnica della Prova Finale
Ingegneria del Software

Autori: **Matteo Cesandri, Alessandro Gheojan**

Matricole: 248170, 247840

Docente: Prof. Mattia Salnitri

Stack: Angular · Spring Boot 3

LEGENDA DOCUMENTAZIONE

1. *INTRODUZIONE AL PROGETTO*
 - 1.1. *Attori del Sistema*
 - 1.2. *Ambito Funzionale*
2. *ARCHITETTURA DEL SISTEMA*
 - 2.1. *Stack Tecnologico*
 - 2.2. *Struttura a Livelli*
 - 2.3. *Organizzazione dei Package*
3. *USE CASE DIAGRAM*
4. *MODELLO DEI DATI*
 - 4.1. *Class Diagram*
 - 4.2. *Gerarchia degli Utenti (SINGLE_TABLE)*
 - 4.3. *Spazi, Stanze e Desk*
 - 4.4. *Prenotazioni e Waitlist*
 - 4.5. *Ticket e Note*
 - 4.6. *Recensioni*
5. *STRUTTURA DETTAGLIATA DEL CODICE*
 - 5.1. *Il Package Config*
 - 5.2. *Il Package Controller*
 - 5.3. *Il Package DTO*
 - 5.4. *Il Package Service*
 - 5.5. *Il Package Model*
 - 5.6. *Il Package Security*
6. *DESIGN PATTERN IMPLEMENTATI*
 - 6.1. *State Pattern: Il Ciclo di Vita del Desk*
 - 6.2. *State Pattern: Il Ciclo di Vita del Ticket*
 - 6.3. *Observer Pattern: Domain Events e Notifiche*
 - 6.4. *Facade Pattern: Le State Machine come Interfaccia Semplice*
 - 6.5. *Repository Pattern con Lock Pessimistico*
 - 6.6. *MapStruct: Mapping DTO Senza Boilerplate*
 - 6.7. *Messaggi Centralizzati: Enum come Repository di Stringhe*
7. *FLUSSI PRINCIPALI DEL SISTEMA*
 - 7.1. *Flusso di Autenticazione*
 - 7.2. *Flusso di Prenotazione*
 - 7.3. *Flusso Ticket di Assistenza*
8. *GESTIONE DELLE NOTIFICHE*
 - 8.1. *Perché SSE invece di WebSocket*
 - 8.2. *Implementazione: NotificationStreamHub e PersistentNotificationService*
 - 8.3. *Sicurezza della Connessione SSE*

9. *SICUREZZA E AUTENTICAZIONE*

- 9.1. *JWT e Autenticazione Stateless*
- 9.2. *AccessControlService: Prevenzione IDOR*
- 9.3. *SecurityConfig: Regole di Accesso*

10. *GESTIONE DEGLI ERRORI*

11. *MECCANISMI TECNICI RILEVANTI*

- 11.1. *Lock Pessimistico e Anti-Double-Booking*
- 11.2. *Lock Ottimistico*
- 11.3. *Generazione Codici Univoci con Retry*
- 11.4. *Orari di Apertura in JSON*
- 11.5. *Seeder Idempotente*

12. *STRATEGIA DI TEST*

- 12.1. *Test Unitari: Controller*
- 12.2. *Test Unitari: Service e Model*
- 12.3. *Test di Integrazione e Concorrenza*
- 12.4. *Test di Sicurezza*

13. *ARCHITETTURA FRONTEND*

- 13.1. *Stack Tecnologico*
- 13.2. *Organizzazione dell'Applicazione*
- 13.3. *Routing, Shell e Navigazione per Ruolo*
- 13.4. *Comunicazione con il Backend*
- 13.5. *Stato delle Pagine e Gestione dei Dati*
- 13.6. *Modelli TypeScript e Programmazione Orientata agli Oggetti*
- 13.7. *Flussi Principali dell'Interfaccia*
- 13.8. *Sessione, Notifiche e Aggiornamenti della UI*
- 13.9. *Interfaccia, Componenti Condivisi e Stile*

14. *CONSIDERAZIONI FINALI*

- 14.1. *Punti di Forza*
- 14.2. *Trade-off e Limitazioni Note*

Introduzione al Progetto

SmartDesk è un sistema di gestione uffici per il coworking realizzato con uno stack applicativo frontend: Angular e backend: REST in Java con Spring Boot 3, progettato per gestire l'intera operatività e logica degli spazi di coworking. L'idea è dunque intuitiva: si ha **un'azienda o un gestore che ha più sedi in diverse città, in ogni sede ci sono stanze e postazioni (desk) da prenotare, e tutta la logistica, prenotazioni, manutenzione, segnalazioni, recensioni, deve essere gestita da un'unica piattaforma coerente.**

Il progetto come tutti i software di buone dimensioni deve rispondere però a problemi ingegneristici reali, in questo caso quelli che qualsiasi sistema di prenotazione deve affrontare: ad esempio come si evita che due utenti prenotino lo stesso desk nello stesso momento? Oppure come notificare in tempo reale un lavoratore quando la sua postazione entra in manutenzione? Come si modella correttamente il ciclo di vita di un guasto, dall'apertura del ticket fino alla dismissione del desk? Queste domande non sono purtroppo solo teoriche, ma necessitano di risvolti pratici ed ognuna ha guidato una scelta architettonica specifica, descritta nei capitoli seguenti.

Da un punto di vista tecnico, SmartDesk espone un'API RESTful completamente documentata tramite OpenAPI/Swagger, utilizza JWT per l'autenticazione stateless e Server-Sent Events per le notifiche in tempo reale. Il backend è pronto per girare sia in ambiente di sviluppo locale (con H2 embedded) sia in produzione (con MySQL 8), semplicemente cambiando il profilo Spring attivo.

NOTA IMPORTANTE: Tutte le immagini del documento sono PNG ad alta risoluzione, nel caso si abbia difficoltà a leggere ingrandire leggermente.

Viste dell'applicazione (Admin, Host, Worker, Technician):

Admin

SmartDesk

Admin

UTENTI

RICHIESTE

SA

SYSTEM ADMINISTR...

DASHBOARD

UTENTI

RICHIESTE

SPAZI

PRENOTAZIONI

LOG

RECENSIONI

NOTIFICHE

OVERVIEW

Dashboard amministratore

UTENTI TOTALI

23

HOST IN ATTESA

2

SPAZI ATTIVI

4

SPAZI ATTESA

2

Gestisci →

Richieste pendenti

Ultime 3 su 4 richieste (2 host - 2 spazi)

VEDI TUTTE →

TIPO	NOME	DETTAGLIO	REGISTRATO	AZIONI
Host	Antonio Sala	antonio.pending@host.com	6 giu 2026, 12:16	GESTISCI
Host	Chiara InAttesa	chiara.inattesa@host.com	4 giu 2026, 16:55	GESTISCI
Spazio	Milano Coworking Space	Milano · Giuseppe Verdi	—	GESTISCI

Host

SmartDesk

Host

PRENOTAZIONI

SEGNALAZIONI

GV

GIUSEPPE VERDI

DASHBOARD

SPAZI

POSTAZIONI

TECNICI

RECENSIONI

SEGNALAZIONI

PRENOTAZIONI

NOTIFICHE

Gestisci postazioni e recensioni.

Panoramica completa dei tuoi spazi e attività

STATO DEGLI SPAZI

SPAZI TOTALI

1

APPROVATI

1

IN ATTESA

0

AZIONI RAPIDE & GESTIONE

I miei spazi

1 spazi gestiti

Postazioni

24 desk totali

Tecnici

1 addetti

Ticket

3 segnalazioni

Recensioni

48 valutazioni

Notifiche

Centro messaggi

Pagina | 4

Worker

SmartDesk

Worker

PRENOTA

PRENOTAZIONI

MR MARIO ROSSI

DASHBOARD

POSTAZIONI

PRENOTAZIONI

SEGNALAZIONI

NOTIFICHE

RECENSIONI

PROFILO

Benvenuto Mario Rossi, hai intenzione di prenotare oggi?

PRENOTAZIONI OGGI

1

PRENOTAZIONI ATTIVE

6

SEGNALAZIONI APERTE

3

NOTIFICHE

0

AGENDA DI OGGI

Milano Central Coworking · Open Space

16:00 – 18:30

Desk 03

AZIONI RAPIDE

Trova postazione
Prenota uno spazio

Prenotazioni
6 prenotazioni attive

Supporto
3 segnalazioni aperte

Notifiche
Centro messaggi

Technician

SmartDesk

Tecnico

IN ATTESA

ASSEGNATE

RG ROBERTO GIALLI

DASHBOARD

IN ATTESA

ASSEGNATE

MANUTENZIONE

NOTIFICHE

PANNELLO TECNICO

Ciao, Roberto Gialli bentornato

Ticket attivi in gestione

2

Monitor flickering

Creato il 03/06/2026 13:16

In corso

Proiettore non rilevato

Creato il 01/06/2026 11:24

In corso

Ticket in coda

3 in attesa

I tuoi ticket

2 attivi

Manutenzione desk

1 uffici

Notifiche

Nessuna

Pagina | 5

1.1 Attori del Sistema

Il sistema distingue quattro ruoli con responsabilità ben separate. Questa separazione si dimostra non solo nelle funzionalità ma si riflette anche direttamente nella struttura degli endpoint, nelle regole di sicurezza e nel modello dei dati.

Ruolo	Identità	Responsabilità principali
Worker	Il lavoratore che prenota	Cerca desk, crea prenotazioni, apre ticket, lascia recensioni, si iscrive in waitlist
Host	Il proprietario dello spazio	Gestisce spazi, desk, stanze, approva riparazioni, risponde alle recensioni
Technician	Il tecnico manutentore	Prende in carico i ticket, aggiorna lo stato, mette i desk in manutenzione
SysAdmin	L'amministratore della piattaforma	Approva host e spazi, modera utenti, consulta log di sistema

Ogni ruolo corrisponde a una sottoclasse di User nel modello dei dati e a un gruppo di endpoint dedicato nel livello controller. La separazione è quindi rigida: un Worker non può mai accedere agli endpoint di gestione dello spazio, e un Host non può operare sui desk di spazi che non gli appartengono. Queste regole sono applicate sia a livello di configurazione Spring Security sia tramite controlli di ownership espliciti nel codice dei service.

1.2 Ambito Funzionale

Le macroaree funzionali del sistema coprono integralmente il ciclo di vita di uno spazio coworking gestito. Di seguito un riepilogo delle aree principali:

- **Autenticazione:** registrazione Worker e Host, login JWT, refresh token, logout con revoca.
- **Gestione Spazi e Desk:** CRUD completo di spazi, stanze e postazioni con orari di apertura, chiusure straordinarie e amenity.
- **Prenotazioni:** creazione con lock pessimistico anti-double-booking, cancellazione, riprogrammazione, terminazione sessione, waitlist automatica.
- **Ticket di Assistenza:** ciclo di vita completo con *stati* *OPEN* → *IN_PROGRESS* → *VERIFYING* → *RESOLVED/CLOSED*, assegnazione tecnico, thread di commenti separati per ruolo.
- **Recensioni:** creazione su prenotazione completata, risposta dell'host e nota dell'admin.

- **Notifiche Real-time:** push SSE (Server-Sent Events) per tutte le azioni rilevanti, con contatore badge aggiornato senza polling.
- **Gestione Piattaforma:** audit log, moderazione utenti, approvazione host e spazi da parte del SysAdmin.

2. Architettura del Sistema

L'architettura di SmartDesk che abbiamo scelto segue il **pattern a tre livelli (three-tier)**, che è probabilmente il pattern più diffuso nei sistemi enterprise Java. L'idea di base è quindi immediata: **ogni livello ha un'unica responsabilità e non può "saltare" i livelli adiacenti. I Controller ricevono le richieste HTTP e delegano, i Service contengono tutta la logica di business, i Repository dialogano con il database.** Questa separazione rende il codice testabile, manutenibile e facile da ragionarci sopra.

Una scelta fatta fin dall'inizio è quella di **organizzare il codice per dominio applicativo** (bounded context) piuttosto che per tipo tecnico. Questo significa che tutto ciò che riguarda le prenotazioni sta in `service/booking`, i ticket in `service/ticket`, e così via. Non si ha un singolo file "BookingService.java" con moltissime righe: si hanno quindi più file scompattati per poter operare condizioni in modo mirato ad esempio BookingCreationService, BookingCancellationService, BookingWaitlistService, ognuno con una responsabilità precisa. Questa granularità rende immediato trovare il codice relativo a una funzionalità specifica e riduce il rischio di introdurre bug quando si modifica una parte del sistema.

2.1 Stack Tecnologico

Di seguito le tecnologie principali che abbiamo utilizzato nel progetto e la motivazione dietro ogni scelta.

Componente	Tecnologia	Motivazione
Runtime	Java 17 + Spring Boot 3	LTS stabile, ecosystem maturo, auto-configurazione avanzata
Sicurezza	Spring Security + JWT HS256	Autenticazione stateless: nessuna sessione lato server, scalabilità orizzontale
Persistenza	Spring Data JPA + Hibernate	ORM trasparente, JPQL type-safe, ottimistic locking integrato
DB sviluppo	H2 (file mode)	Zero configurazione, compatibile MySQL, ideale per test e sviluppo locale
DB produzione	MySQL 8	Affidabile, vincoli FK, ottimo con query complesse e lock pessimistici
Mapping DTO	MapStruct	Codice generato a compile-time, nessuna reflection a runtime, type-safe
Notifiche RT	Server-Sent Events (SSE)	Push server→client leggero, nativo HTTP/1.1, senza overhead WebSocket

Componente	Tecnologia	Motivazione
Documentazione API	SpringDoc / OpenAPI 3	Genera Swagger UI automaticamente da annotazioni, zero manutenzione manuale
Boilerplate	Lombok	Elimina getter/setter/builder/logger ripetitivi a compile-time

2.2 Struttura a Livelli

Il flusso di una richiesta HTTP attraversa sempre i tre livelli nell'ordine, senza eccezioni. Prendiamo come esempio la creazione di una prenotazione: il `WorkerBookingController` riceve la richiesta POST, valida il body con `@Valid` (delegando a Spring la verifica dei campi obbligatori e dei formati), poi chiama `BookingCreationService` passando solo ciò che gli serve. Il service si occupa di tutta la logica: controlla gli orari di apertura dello spazio, acquisisce il lock pessimistico sul desk, verifica la disponibilità, e infine chiama il repository per salvare. Il repository esegue la query SQL/JPQL e restituisce il risultato al service, che lo trasforma in un DTO e lo ritorna al controller, che lo serializza in JSON.

Questo flusso lineare potrebbe sembrare macchinoso, ma è una buona pratica che ci fornisce un enorme vantaggio dal momento che è **facilissimo testare ogni livello in isolamento, e quando qualcosa va storto è immediato capire in quale livello il problema si trova per correggerlo quanto prima**.

2.3 Organizzazione dei Package

La struttura dei package rispecchia fedelmente l'architettura a livelli e l'organizzazione per dominio. Di seguito una descrizione di ogni package e del suo ruolo nel sistema.

Package	Contenuto e ruolo
config/bootstrap	Seeder idempotente: AdminSeeder, DemoDataSeeder, BookingSeeder, ReviewSeeder, TicketSeeder
config/web	SecurityConfig, JsonConfig, SwaggerConfig, CORS, tutta la configurazione infrastrutturale
controller/	Controller REST divisi per ruolo: admin, auth, booking, desk, host, profile, space, technician, worker
dto/	Data Transfer Objects per input e output di ogni API, divisi per dominio
event/	Domain events: BookingReleasedEvent, BookingCancelledByHostNoticeEvent

Package	Contenuto e ruolo
exception/	Eccezioni di dominio + RestExceptionHandler globale
mapper/	Interfacce MapStruct per conversione Entity↔DTO
model/	Entità JPA organizzate per sotto-dominio
repository/	Repository Spring Data JPA con query JPQL custom
security/filter	JwtAuthenticationFilter: intercetta ogni richiesta e popola il SecurityContext
service/	Logica di business organizzata per dominio
support/	Utility cross-cutting: ApiErrorSupport, TextValidation, TimeIntervalUtils, CodeUtils
util/	Policy, messaggi enum centralizzati, validatori custom Bean Validation

Il diagramma seguente mostra le dipendenze tra i package principali. Si nota subito che le dipendenze seguono sempre verso il basso (dal controller verso il service verso il repository e il model) e non ci sono dipendenze circolari.

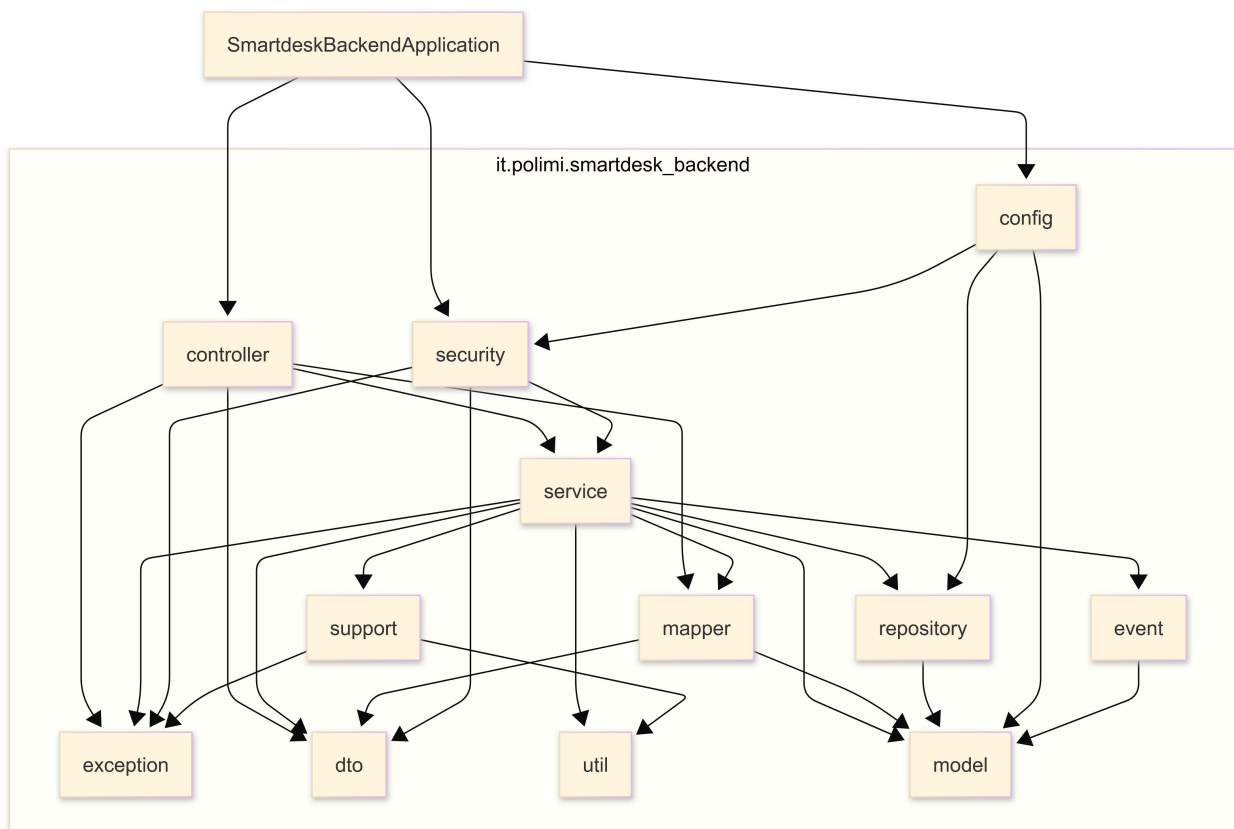


Figura 2.1 — Diagramma delle dipendenze tra package

3. Use Case Diagram

Il diagramma seguente mostra l'insieme completo degli use case del sistema, raggruppati per area funzionale e colorati per attore. È un buon punto di partenza per capire la portata del progetto e le azioni che possono compiere i vari attori, prima di entrare nei dettagli implementativi.

Gli attori principali già precedentemente menzionati sono quattro, Worker, Host, Technician, SysAdmin, interagiscono con il sistema attraverso use case distinti, con alcune sovrapposizioni dove più ruoli condividono le stesse funzionalità. Per esempio, sia Worker che Host possono visualizzare le notifiche e aggiungere note a un ticket, ma con visibilità diverse (un Worker vede solo i propri ticket, l'Host vede quelli di tutti i desk del suo spazio).

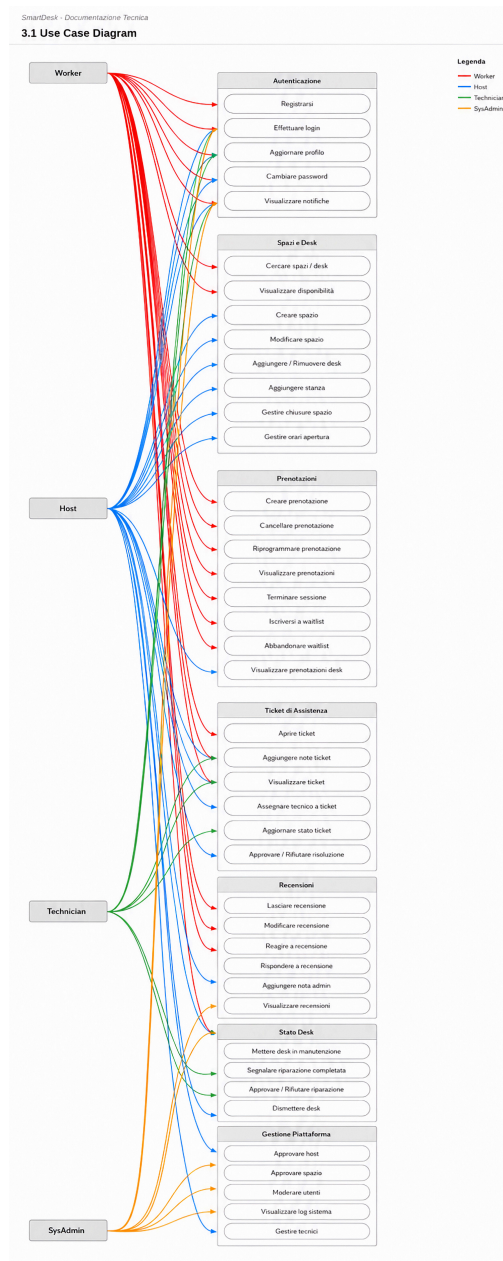


Figura 3.1 — Use Case Diagram completo (Worker=rosso, Host=blu, Technician=verde, SysAdmin=arancio)

Le aree funzionali coperte sono sette: Autenticazione, Spazi e Desk, Prenotazioni, Ticket di Assistenza, Recensioni, Stato Desk, Gestione Piattaforma. Ogni area è trattata in dettaglio nelle sezioni successive, con i flow chart che mostrano i casi d'uso più complessi.

4. Modello dei Dati

4.1 Class Diagram

Il class diagram mostra tutte le entità JPA e le relazioni tra loro. È il cuore del modello dei dati, e capire bene questo diagramma è fondamentale per capire come funziona il resto del software.

Le entità principali sono: User (con gerarchia Worker/Host/Technician/SysAdmin), Space, Room, Desk, Booking, WaitlistEntry, Ticket, Review, Notification, RefreshToken. Ogni entità ha un ID auto-generato e, dove necessario, un campo version per il lock ottimistico.

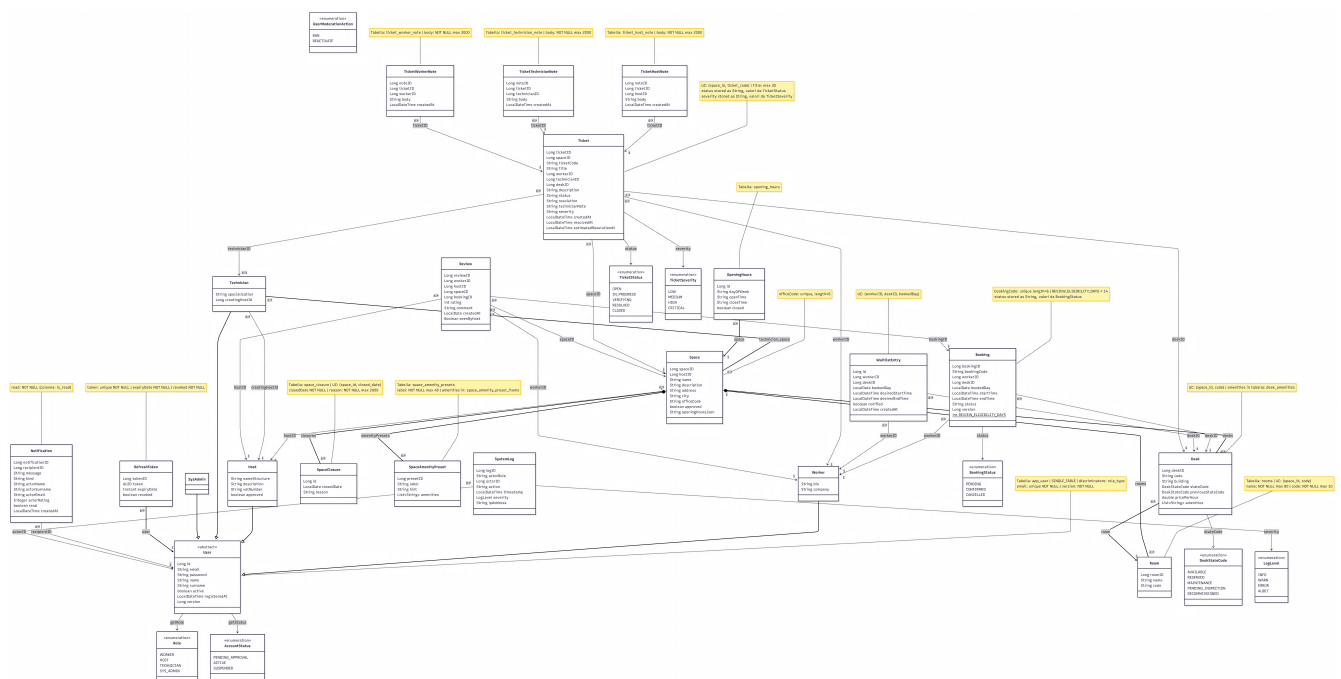


Figura 4.1 — Class Diagram completo delle Entità JPA

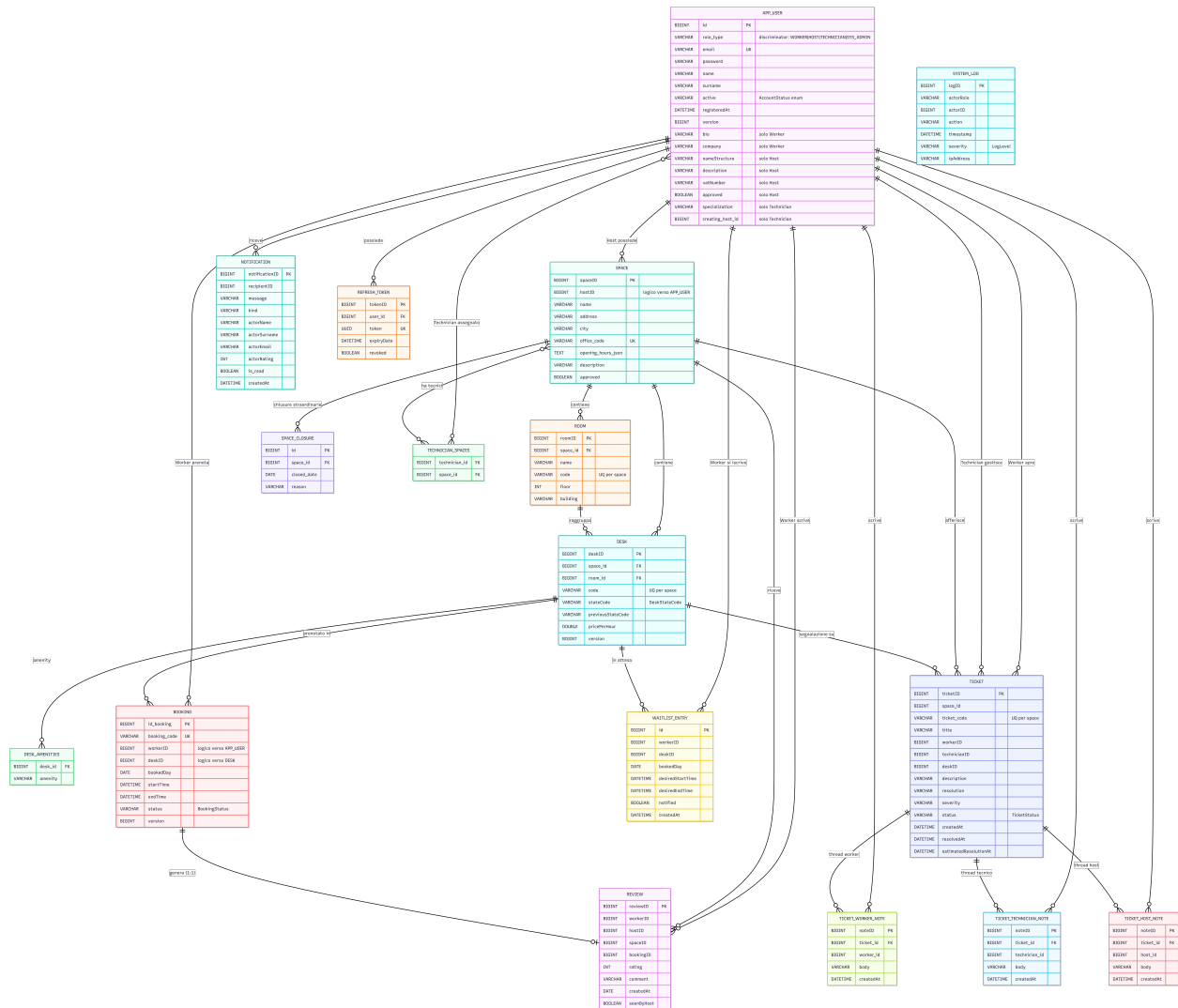


Figura 4.2 — Diagramma ER delle relazioni tra le entità del database

4.2 Gerarchia degli Utenti (SINGLE_TABLE)

Cominciamo con il sottolineare che tutti gli utenti vivono nella stessa tabella `app\_user` nel database. JPA utilizza una colonna discriminatore (`role\_type`) per distinguere Worker, Host, Technician e SysAdmin. La scelta di SINGLE_TABLE semplifica il login, una sola SELECT per email, e le operazioni amministrative.

Perché SINGLE_TABLE? Con questa strategia il login richiede una sola SELECT filtrata per email, senza JOIN tra tabelle. Le colonne specifiche di Host (vatNumber, approved) o Technician (specialization) sono NULL per gli altri ruoli: un compromesso accettabile per il volume previsto del sistema SmartDesk.

La classe User è astratta e contiene i campi condivisi ossia: id, email, password (BCrypt), name, surname, active (boolean), registeredAt, version (per il lock ottimistico). I sotto-tipi aggiungono campi specifici del loro ruolo. Il Worker aggiunge ad esempio bio e company, per tracciare informazioni sul lavoratore che prenota. L'Host aggiunge nameStructure, vatNumber, description e il campo approved, che deve essere impostato a true dal SysAdmin prima che l'host possa realmente operare. Il Technician aggiunge specialization e creatingHostId, perché ogni tecnico viene creato e gestito dall'Host a cui appartiene. Il SysAdmin infine non ha campi extra, ma ha accesso completo a tutte le funzionalità della piattaforma.

4.3 Spazi, Stanze e Desk

La gerarchia **Space** → **Room** → **Desk** rispecchia fedelmente la struttura fisica di un **coworking**. Uno Space rappresenta un'intera sede (edificio, città, indirizzo). Ogni Space può contenere più Room, che sono stanze o aree logiche all'interno dello spazio (ad esempio "Sala A", "Piano 2", "Open Space"). Ogni Room può contenere più Desk, che sono le singole postazioni prenotabili.

Una decisione progettuale interessante riguarda la gestione degli orari di apertura: invece di creare una tabella separata con righe per ogni giorno della settimana, abbiamo serializzato gli orari in una colonna TEXT in formato JSON. Questo approccio ci permette allora di modificare la struttura degli orari (aggiungendo magari fasce orarie multiple per giorno) senza dover migrare il database.

Entità	Campi chiave	Note
Space	spaceId, name, city, address, officeCode, approved, openingHoursJson	openingHoursJson è un TEXT JSON: consente di modificare la struttura senza migrazioni DB
Room	roomId, name, floor, officeCode, building	Raggruppa desk per area fisica; officeCode ereditato dallo Space
Desk	deskId, code, stateCode, amenities, pricePerHour	code è UNIQUE per spazio; amenities è List<String> persistito come element-collection
SpaceClosure	closedDate, reason	Chiusura straordinaria: verificata prima di ogni prenotazione

4.4 Prenotazioni e Waitlist

Booking modella una prenotazione con tutti i metadati necessari. Una scelta un po' insolita che abbiamo fatto è quella di salvare i riferimenti al worker e al desk come ID semplici (Long) piuttosto che come oggetti JPA con la relazione @ManyToMany. Questo ha l'obiettivo di **evitare caricamenti**

superflui: quando vado a leggere una prenotazione, non ho bisogno dei dettagli completi del desk o dell'utente, mi basta l'ID. Solo quando servono i dettagli completi (ad esempio per la vista "le mie prenotazioni") si fa un JOIN esplicito.

WaitlistEntry memorizza invece la richiesta di un Worker di essere notificato quando si libera un desk in un certo giorno. Il campo `notified` evita notifiche duplicate: una volta che la notifica è stata inviata, l'entry viene marcata come notified e non viene più considerata.

Entità	Campi chiave	Note
Booking	bookingId, bookingCode, workerID, deskID, bookedDay, startTime, endTime, status, version	status: PENDING / CONFIRMED / CANCELLED. bookingCode da CodeUtils (6 chr)
WaitlistEntry	workerID, deskID, bookedDay, notified, desiredStart/EndTime	Ordinata per createdAt ASC (FIFO). notified=false finché non viene inviata la notifica

4.5 Ticket e Note

Ticket modella ovviamente una segnalazione di guasto o di richiesta di manutenzione. Il sistema di note è una delle parti più interessanti dal punto di vista del design: invece di avere un unico thread di commenti visibile a tutti, abbiamo **tre thread separati per ruolo, TicketWorkerNote, TicketTechnicianNote, TicketHostNote**. Questo simula il comportamento di molti sistemi di ticketing reali, dove worker, tecnici e host comunicano su canali leggermente diversi, con diversi livelli di formalità e visibilità.

Il ciclo di vita del ticket è gestito tramite una state machine, descritta in dettaglio nella sezione sui design pattern (la trova più avanti). I possibili stati e le transizioni sono:

Stato	Significato	Transizioni
OPEN	Ticket appena aperto dal Worker	→ IN_PROGRESS (host assegna tecnico)
IN_PROGRESS	Tecnico al lavoro	→ VERIFYING (tecnico segnala completamento)
VERIFYING	In attesa di approvazione host	→ RESOLVED (host approva) oppure → IN_PROGRESS (host rifiuta)
RESOLVED	Risolto, desk torna AVAILABLE	Stato finale normale
CLOSED	Chiuso per dismissione desk	Stato terminale alternativo (bypass VERIFYING)

4.6 Recensioni

Review è collegata 1:1 a una Booking completata. L'entità mantiene rating (1-5), commento, data di creazione, i riferimenti a worker, host, space e booking, e il flag seenByHost che dice se l'host ha già letto la recensione. La risposta dell'host e la nota dell'admin sono gestite dai service dedicati (HostReviewService, AdminReviewService) insieme alle relative notifiche al Worker.

5. Struttura Dettagliata del Codice

In questa sezione cerchiamo di entrare nel dettaglio della struttura del codice, descrivendo come ogni package è organizzato internamente e quale filosofia guida le scelte implementative. L'obiettivo non quindi è commentare ogni singola riga, ma cercare di spiegare il "perché" dietro le strutture che abbiamo scelto.

5.1 Il Package `config`

Il package config è diviso in due sotto-package: bootstrap e web. Questa separazione non è casuale: bootstrap contiene tutto ciò che riguarda l'inizializzazione dei dati (i seeder), mentre web contiene la configurazione dell'infrastruttura applicativa (sicurezza, JSON, Swagger, CORS).

I seeder del package bootstrap meritano invece un discorso a parte. **La classe DemoDataSeeder legge un file JSON (seed-data.json) all'avvio dell'applicazione in profilo dev e popola il database** con dati di esempio: worker, host, tecnici, spazi, desk, prenotazioni, ticket, recensioni. La caratteristica più importante è l'idempotenza: ossia ogni record viene creato solo se non esiste già. Questo significa che si può riavviare l'applicazione quante volte si vuole senza creare duplicati.

SecurityConfig è la classe più delicata del package web. Definisce la catena dei filtri di sicurezza, le regole di accesso per ogni gruppo di endpoint, e la configurazione CORS. La catena dichiara come pubblici solo gli endpoint di registrazione, login, refresh e Swagger; tutto il resto sotto /api/** richiede un token JWT valido. Il JwtAuthenticationFilter viene inserito prima del filtro di autenticazione standard di Spring per intercettare e validare il token JWT ad ogni richiesta, mentre il controllo del ruolo viene fatto a livello di service tramite AccessControlService.

5.2 Il Package `controller`

I controller sono organizzati per ruolo e dominio. Questa struttura rispecchia esattamente la struttura degli endpoint esposti dall'API: `/api/workers/\*\*` è gestito dai controller nel sotto-package worker, `/api/hosts/\*\*` da quelli in host, e così via.

Ogni controller è deliberatamente "sottile": non contiene pertanto logica di business. La sua responsabilità è limitata a tre cose: ricevere la richiesta, validare l'input nel body della richiesta con @Valid, e delegare al service appropriato. Il risultato è che leggendo allora un controller si capisce immediatamente cosa fa quell'endpoint senza dover seguire catene di logica. Per esempio, nel caso di `WorkerBookingController.createBooking()` questo chiama semplicemente `bookingCreationService.createBooking(request, authenticatedUser)` e ritorna il risultato: tutto il resto è nel service.

Per quanto riguarda la gestione dell'utente autenticato, invece di estrarre l'ID utente dal token JWT in ogni metodo, abbiamo creato **un resolver personalizzato che inietta automaticamente un oggetto AuthenticatedUser nei parametri del metodo quando annotato con**

@AuthenticationPrincipal. Questo oggetto contiene l'ID utente, l'email e il ruolo, e viene costruito dal JwtAuthenticationFilter ad ogni richiesta.

5.3 Il Package `dto`

I DTO (Data Transfer Objects) rappresentano il contratto dell'API con il mondo esterno (view). Sono deliberatamente separati dalle entità JPA del package model: questa separazione è fondamentale perché l'entità JPA rispecchia la struttura del database, mentre il DTO rispecchia ciò che il client ha bisogno di vedere (o mandare).

Per esempio, l'entità Booking contiene un riferimento all'ID del desk come Long, ma il BookingDTO che il client riceve contiene il codice del desk, il nome della stanza e della sede, l'indirizzo. Queste informazioni vengono assemblate dal mapper in fase di conversione.

I DTO di request sono annotati con Bean Validation (`@NotNull`, `@Size`, `@FutureOrPresent` ecc.) per garantire che i dati in ingresso siano sempre validi prima di arrivare al service. I DTO di response sono semplici classi con i campi necessari, senza logica.

5.4 Il Package `service`

Il package service è il fulcro operativo dell'applicazione. Qui risiede la logica di business, organizzata per dominio con una granularità fine. Invece di avere un generico "BookingService" con tutti i metodi relativi alle prenotazioni, si ha per esempio:

- **BookingCreationService:** crea e riprogramma prenotazioni. Contiene la logica del lock pessimistico, la verifica degli orari di apertura, e i controlli anti-double-booking.
- **BookingCancellationService:** cancella prenotazioni verificando l'ownership. Pubblica l'evento BookingReleasedEvent dopo il commit.
- **BookingWaitlistService:** gestisce iscrizione e notifica waitlist con REQUIRES_NEW per eseguire in transazione separata.
- **BookingEndSessionService:** termina anticipatamente una sessione aggiornando endTime.
- **DeskAvailabilityService:** ricerca desk disponibili con filtri multipli, generando una griglia di slot da 30 minuti.

Questa granularità ha un costo dal punto di vista del numero di classi, ma ha comunque vantaggi significativi: ad esempio ogni classe ha una responsabilità chiara e può essere testata in isolamento, i metodi sono brevi e leggibili, e aggiungere una nuova funzionalità non richiede di modificare una classe già complessa.

5.5 Il Package `model`

Le entità JPA del package model rappresentano il dominio dell'applicazione mappato sul database. Sono organizzate in sotto-package per dominio: booking, notification, review, space, ticket, user.

Abbiamo cercato di mantenere le entità il più possibile "pure", cioè senza logica applicativa al loro interno. Le entità contengono solo i campi, le relazioni JPA, e al massimo qualche metodo di convenienza semplice. Tutta la logica di business sta nei service.

Un'eccezione a questa regola riguarda **gli enum di stato (DeskStateCode, TicketStatus, BookingStatus)**: questi sono **definiti direttamente nel package model** perché sono parte integrante del contratto del dominio e vengono usati sia nelle entità che nei service.

5.6 Il Package `security`

Il package security contiene il filtro JWT e le utility di autenticazione. Il JwtAuthenticationFilter è il componente più critico: viene eseguito ad ogni richiesta HTTP (eccetto gli endpoint pubblici) e ha la responsabilità di estrarre il token JWT dall'header Authorization, validarlo, e popolare il SecurityContext di Spring con l'utente autenticato.

La validazione del token include più controlli: la firma HS256 deve essere valida (usando la chiave segreta configurata), il token non deve essere scaduto, e l'utente associato deve essere attivo nel database. Se qualsiasi controllo fallisce, il filtro restituisce 401 Unauthorized senza propagare la richiesta al controller.

6. Design Pattern Implementati

Nel progetto abbiamo cercato di utilizzare diversi design pattern in modo da ingegnerizzarne quanto più possibile l'architettura. Ogni pattern inserito è mirato a risolvere un problema concreto che abbiamo riscontrato durante lo sviluppo. In questa sezione andiamo quindi a menzionare i vari pattern utilizzati con il problema specifico che ha risolto e come è stato implementato.

6.1 State Pattern: Il Ciclo di Vita del Desk

Il problema di partenza era semplice da enunciare ma comunque difficile da gestire senza una logica appropriata: abbiamo scelto che un desk possa trovarsi in **cinque stati distinti** (AVAILABLE, RESERVED, MAINTENANCE, PENDING_INSPECTION, DECOMMISSIONED) e le operazioni attuabili permesse cambiano completamente a seconda dello stato corrente in cui si trova. Mettere ad esempio in manutenzione un desk che è già in manutenzione non ha senso. Oppure prenotare un desk DECOMMISSIONED sarebbe un errore grave.

Senza lo State Pattern, il codice del service sarebbe pieno di blocchi condizionali del tipo "se lo stato è X, e l'operazione è Y, allora fai Z, altrimenti lancia un'eccezione". Con cinque stati e una dozzina di operazioni, questo porta a una combinatoria eccessiva di if/else difficile da seguire e ancora più difficile da modificare senza introdurre bug o ulteriori problemi. Qui sotto uno schema delle transizioni possibili:

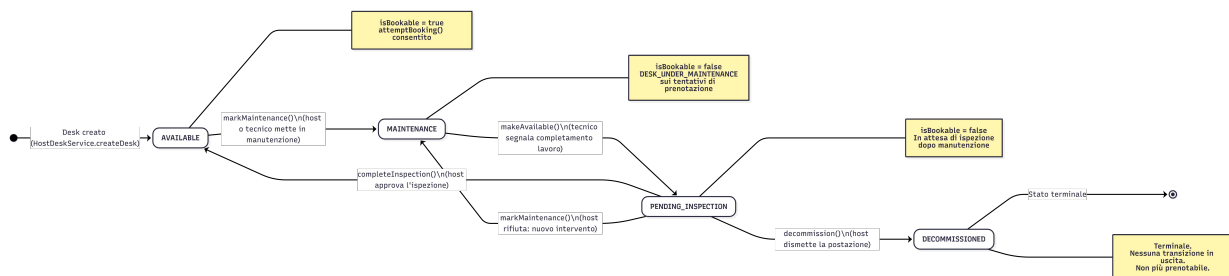


Figura 6.1.A — State diagram del ciclo di vita del Desk

La soluzione è stata pertanto **implementare un'interfaccia DeskState** con i metodi che ogni stato deve saper gestire, e una **classe concreta per ogni stato**. Ogni classe implementa solo le transizioni che ha senso fare da quello stato, e lancia una BusinessException con un messaggio che sia chiaro per tutte le altre. **Il DeskStateMachine funge invece da facciata**: il service non deve mai sapere in che stato si trova il desk, chiama semplicemente il metodo sulla macchina a stati e questa si occupa di delegare allo stato corretto, grosso modo un wrapper della logica.

```
// Interfaccia comune a tutti gli stati
public interface DeskState {
    void attemptBooking(Desk desk);
    void onEnterMaintenance(Desk desk);
    void onExitMaintenance(Desk desk);
    void decommission(Desk desk);
}

// Implementazione per lo stato AVAILABLE
public class AvailableState implements DeskState {
    public void attemptBooking(Desk desk) { /* ok, desk prenotabile */ }
    public void onEnterMaintenance(Desk desk) {
        /* transizione permessa, la state machine aggiorna lo stato */
    }
    public void onExitMaintenance(Desk desk) {
        throw new BusinessException("Desk non in manutenzione");
    }
}
```

Il vantaggio concreto si dimostra nel momento in cui si desidera ad esempio aggiungere un nuovo stato (ipotizziamo un futuro stato SUSPENDED). Questo richiederebbe solo di creare una nuova classe che implementa DeskState e aggiornare DeskStateMachine: nessuna modifica dunque ai service esistenti. Qui sotto il principio Open/Closed del desk in azione.

Stato	Operazioni permesse	Operazioni negate
AVAILABLE	attemptBooking, onEnterMaintenance	onExitMaintenance, decommission
RESERVED	Solo uso interno sistema	Tutte le operazioni utente
MAINTENANCE	onExitMaintenance, decommission	attemptBooking, onEnterMaintenance
PENDING_INSPECTION	inspectOk → AVAILABLE, rejectInspection → MAINTENANCE	attemptBooking, onEnterMaintenance
DECOMMISSIONED	Nessuna (stato terminale)	Tutte

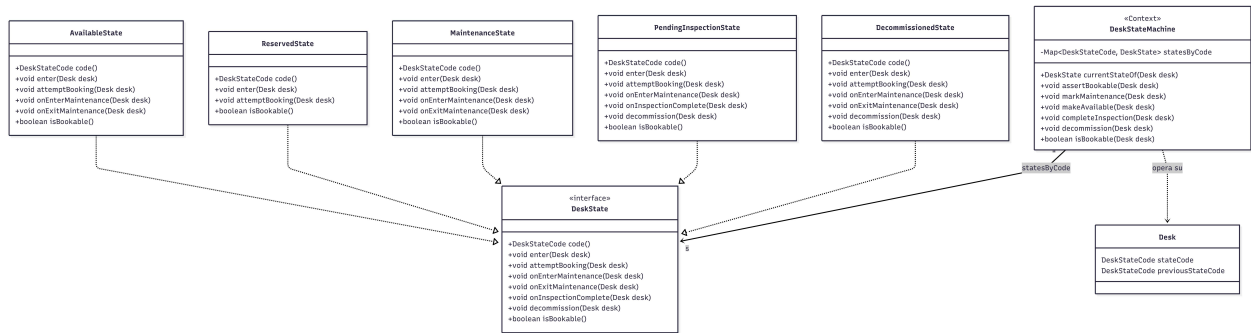


Figura 6.1.B — UML Design State Pattern Desk

6.2 State Pattern: Il Ciclo di Vita del Ticket

Lo stesso principio si applica ugualmente ai ticket di assistenza, con una macchina a stati analoga ma con transizioni diverse. La **TicketStateMachine** è la facciata che espone i metodi assignTechnician, verify, approve, reject, resolve, close. Ogni stato concreto (**OpenTicketState**, **InProgressTicketState**, **VerifyingTicketState**, **ResolvedTicketState**, **ClosedTicketState**) implementa l'interfaccia **TicketState**.

Un caso interessante è invece la transizione "dismiss-desk": quando l'host decide di dismettere un desk invece di ripararlo, il ticket va direttamente in CLOSED bypassando gli stati VERIFYING e RESOLVED. Questa transizione è implementata solo nello stato IN_PROGRESS, e ClosedTicketState non permette nessuna ulteriore transizione (stato terminale).

Il grande vantaggio dello State Pattern nei ticket è lo stesso che per i desk: ossia tentare ad esempio una transizione non permessa (ad esempio approvare un ticket che è già RESOLVED) lancia un'eccezione con un messaggio chiaro, senza che il service debba controllare esplicitamente lo stato corrente. In questo modo il codice del TicketService diventa molto più leggibile: ogni metodo chiama semplicemente la macchina a stati e gestisce l'eccezione solo se necessario.

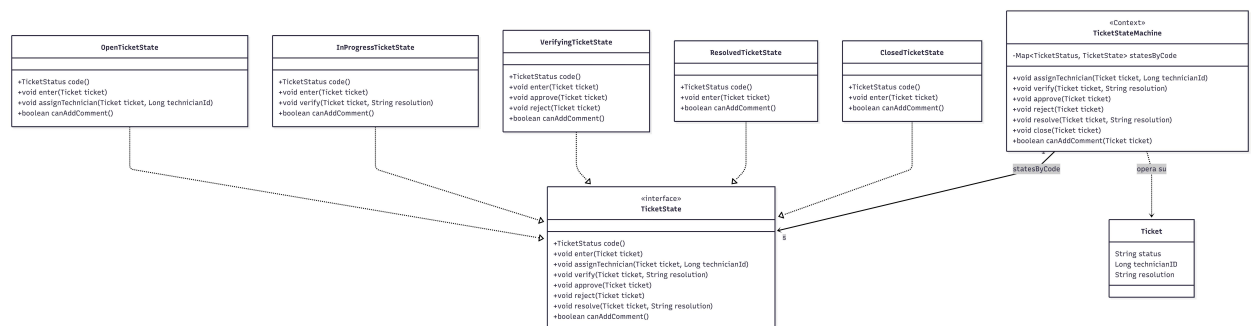


Figura 6.2 — UML Design State Pattern Ticket

6.3 Observer Pattern: Domain Events e Notifiche

Il pattern **Observer** è alla base del sistema di notifiche. Quando una prenotazione viene cancellata, il **BookingCancellationService** pubblica allora un **BookingReleasedEvent** tramite **ApplicationEventPublisher** di Spring. A questo punto **due listener** ascoltano questo evento: viene chiamato **BookingWaitlistObserver** (che notifica i Worker in waitlist) e **BookingCancelledByHostNoticeObserver** (che notifica il Worker della cancellazione da parte dell'host).

La cosa più importante, e che merita forse una spiegazione più dettagliata, è l'uso di **AFTER_COMMIT** nella fase di ascolto dell'evento. Da osservare che questo non è un dettaglio implementativo minore, ma è fondamentale per la correttezza del sistema. Vediamo perché.

Perché AFTER_COMMIT è fondamentale: se si usasse un listener normale, la notifica partirebbe prima del commit della transazione. In caso di rollback (errore DB, vincolo violato), il Worker riceverebbe una notifica per una prenotazione che in realtà non esiste. Con AFTER_COMMIT questo scenario non può verificarsi: la notifica viene inviata solo dopo che la transazione è avvenuta con successo.

Il **BookingWaitlistService** usa invece **@Transactional(propagation = REQUIRES_NEW)** perché viene chiamato fuori da qualsiasi transazione attiva (siamo infatti in fase **AFTER_COMMIT**). **Senza questa annotazione, le query di lock del desk fallirebbero** poiché ovviamente **non ci sarebbe una transazione attiva in cui acquisire il lock**.

```
// Pubblicazione evento dopo il salvataggio
@Transactional
public void removeBooking(Long bookingId) {
    Booking booking = bookingRepo.findById(bookingId).orElseThrow(...);
    booking.cancel();
    bookingRepo.save(booking);
    // Evento pubblicato: i listener lo ricevono DOPO il commit
    eventPublisher.publishEvent(new BookingReleasedEvent(deskId, bookedDay, freedStart, freedEnd));
}

// Listener waitlist — eseguito AFTER_COMMIT
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void onEvent(BookingReleasedEvent event) {
    waitlistService.handleReleasedSlot(event.getDeskId(), event.getBookedDay(), ...);
}
```

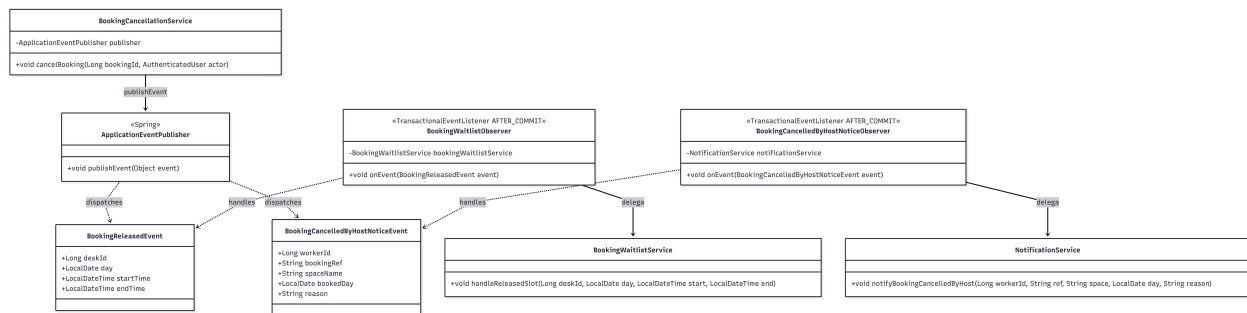


Figura 6.3 — UML Design Observer Pattern

6.4 Facade Pattern: Le State Machine come Interfaccia Semplice

Il Facade Pattern, questo è strettamente legato all'implementazione dello State Pattern. Difatti DeskStateMachine e TicketStateMachine non sono solo implementazioni dello State Pattern: fungono anche da Facade, cioè espongono un'interfaccia semplificata che nasconde la complessità interna della selezione dello stato concreto.

Il service non deve mai sapere quali classi concrete implementano gli stati del desk o del ticket. Chiama semplicemente `deskStateMachine.markMaintenance(desk)` o `ticketStateMachine.approve(ticket)` e la macchina a stati si occupa di tutto il resto: selezionare lo stato corrente, delegare all'implementazione corretta, aggiornare lo stato nel caso di transizione avvenuta.

Questo disaccoppiamento è importante: se domani ad esempio decidessimo di cambiare come viene selezionato lo stato concreto (ipoteticamente usando una mappa invece di uno switch), il service non dovrebbe cambiare nulla.

6.5 Repository Pattern con Lock Pessimistico

Non un vero e proprio pattern ma valeva la pena sottolinearlo, tutti i repository estendono JpaRepository di Spring Data, ma la cosa più interessante è l'uso del lock pessimistico nella creazione delle prenotazioni. La query più critica di tutto il sistema e quindi maggiormente degna di nota è sicuramente `lockByDeskIdForUpdate()`:

```

@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT d FROM Desk d WHERE d.deskId = :id")
Optional<Desk> lockByDeskIdForUpdate(@Param("deskId") Long deskId);
  
```

Questa query si traduce in una `SELECT ... FOR UPDATE` su MySQL. Prima di verificare la disponibilità e salvare una prenotazione, si blocca la riga del desk: qualsiasi altra transazione concorrente che tenti lo stesso slot aspetta il rilascio del lock. Il risultato è zero doppie prenotazioni anche sotto carico elevato.

Oltre al lock pessimistico, abbiamo due query di overlap nel BookingRepository per verificare che non ci siano conflitti:

- **countDeskOverlapping()**: conta le prenotazioni attive sullo stesso desk che si sovrappongono alla fascia oraria richiesta.
- **countWorkerOverlapping()**: conta le prenotazioni attive dello stesso worker che si sovrappongono, per evitare che un worker prenoti due desk in contemporanea.

Entrambe le query usano la condizione classica per il rilevamento di overlap temporali: `startTime < endTimeRichiesto AND endTime > startTimeRichiesto`. Se il conteggio è > 0, la prenotazione viene rifiutata con 409 CONFLICT.

6.6 MapStruct: Mapping DTO Senza Boilerplate

Per la conversione tra entità JPA e DTO è stato utilizzato invece MapStruct, una libreria di code generation che genera il codice di mapping a compile-time. Questo approccio è tal modo preferibile rispetto alla reflection a runtime (usata da librerie come ModelMapper) per due motivi principali: le performance sono nettamente migliori, e gli errori di mapping vengono rilevati a compile-time invece che a runtime.

Le interfacce di mapping che abbiamo definito sono BookingDtoMapper, ReviewResponseMapper, SpaceMapper e TechnicianMapper. Per i mapping che richiedono dati da più sorgenti (ad esempio BookingDTO richiede informazioni da Booking, Desk e User) MapStruct supporta metodi con più parametri di input, ognuno dei quali contribuisce campi al risultato.

6.7 Messaggi Centralizzati: Enum come Repository di Stringhe

Tutti i messaggi di errore e di notifica sono definiti in enum centralizzati: BookingMessage, TicketMessage, SpaceMessage, AuthMessage, ReviewMessage, AdminMessage, ResourceMessage, HttpMessage. Ogni enum implementa allora un contratto comune con il metodo text().

Questo pattern elimina completamente le stringhe duplicate nel codice. Non c'è pertanto nessun rischio che "Prenotazione non trovata" compaia scritto in dieci modi diversi in dieci punti diversi del codebase. Tutte le eccezioni fanno quindi riferimento a uno di questi enum, e il messaggio è sempre coerente. Come bonus da evidenziare, un'eventuale futura internazionalizzazione richiederebbe solo di modificare questi enum.

```
public enum BookingMessage {  
    DESK_ALREADY_BOOKED("La postazione risulta già occupata nella fascia oraria selezionata."),  
    WORKER_SLOT_OVERLAP("Hai già una prenotazione che si sovrappone a questa fascia."),  
    BOOKING_CANNOT_CANCEL_IN_PROGRESS("La prenotazione è già iniziata: non puoi annullarla.");  
  
    private final String text;  
    public String text() { return text; }  
}
```


7. Flussi Principali del Sistema

In questa sezione andiamo ad illustrare i tre flussi principali del sistema con il supporto dei diagrammi di flusso. Ogni flusso rappresenta un percorso reale che un utente può compiere all'interno della piattaforma, con tutti i casi d'errore e potenziali varianti.

7.1 Flusso di Autenticazione

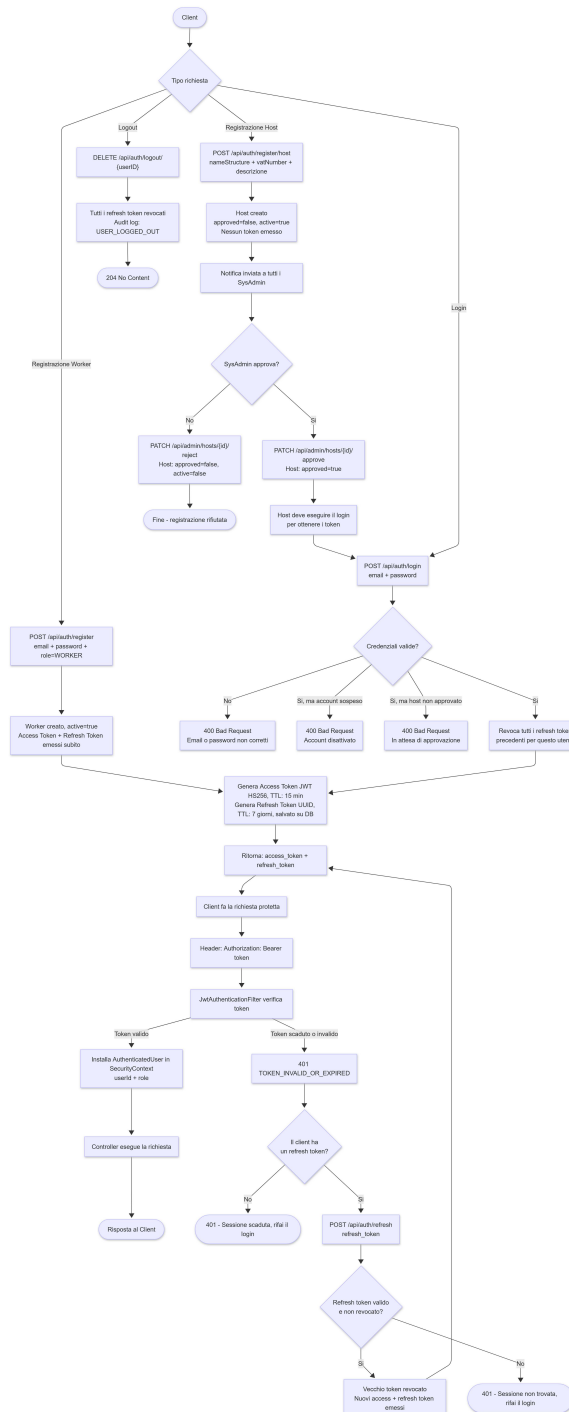


Figura 7.1 — Flowchart del flusso di autenticazione e refresh token

7.2 Flusso di Prenotazione

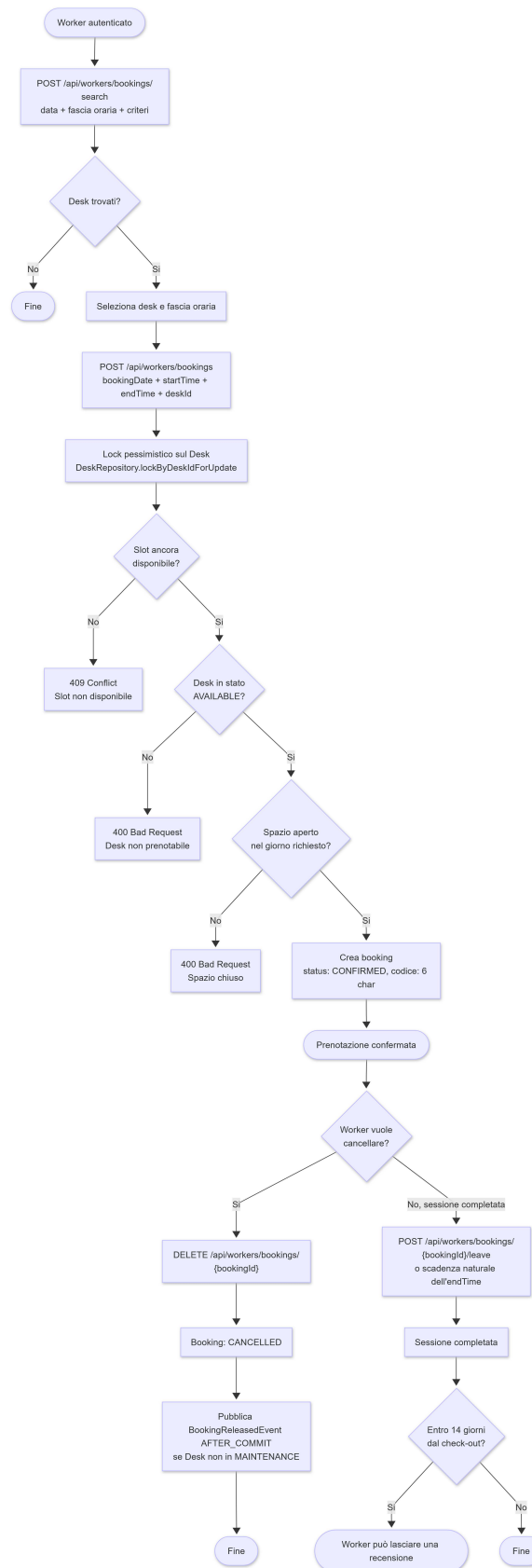


Figura 7.2 — Flowchart del flusso di prenotazione

7.3 Flusso Ticket di Assistenza

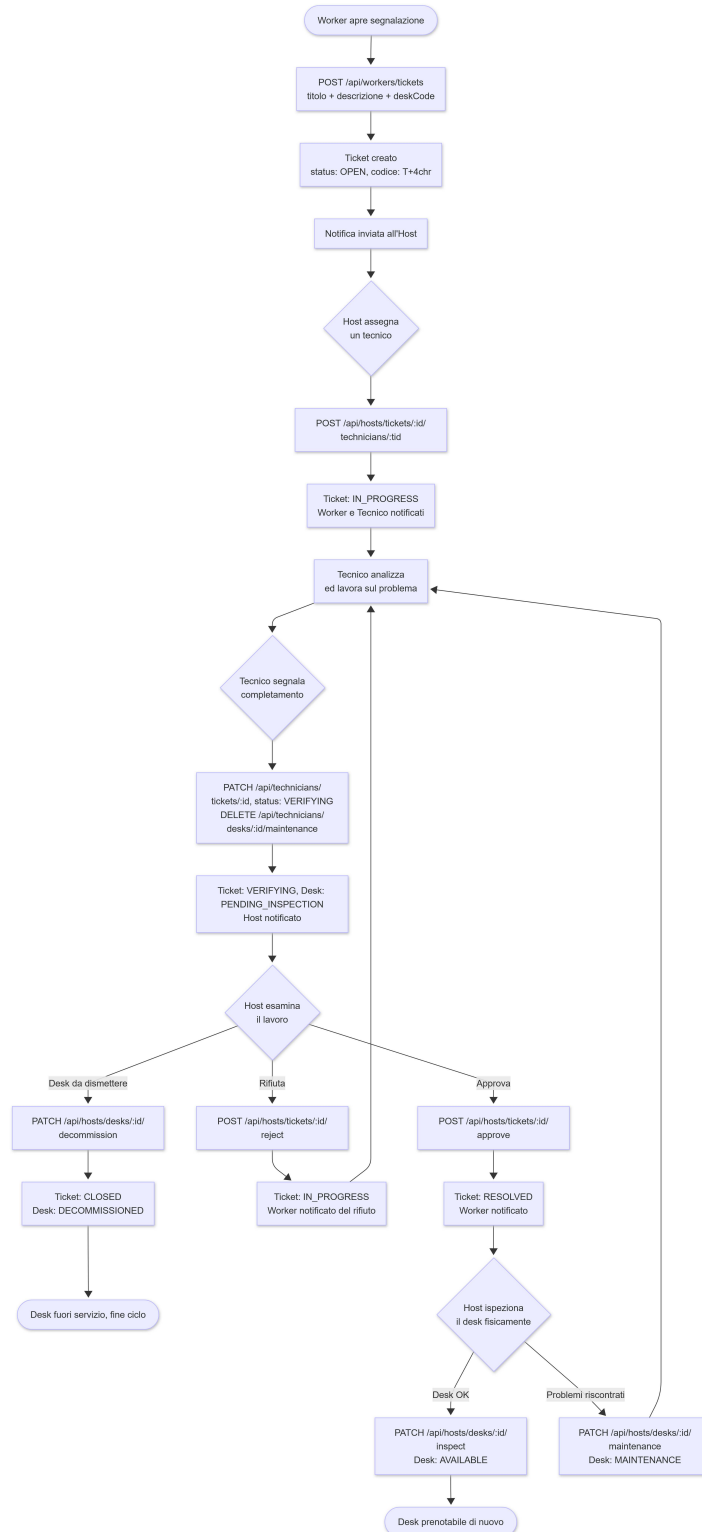


Figura 7.3 — Flowchart del flusso ticket (apertura → risoluzione/dismissione)

8. Gestione delle notifiche

Le notifiche in tempo reale sono uno dei requisiti più tangibili del sistema dal punto di vista della user experience. Per implementarle abbiamo scelto di evitare il polling continuo al backend e di adottare invece le SSE, che permettono al server di inviare eventi al client in modo unidirezionale senza che il client debba interrogare continuamente l'API.

8.1 Perché SSE invece di WebSocket

Quando si parla di comunicazione in tempo reale tra client e server, WebSocket è spesso la prima tecnologia che viene in mente. Tuttavia, per il caso d'uso specifico di SmartDesk, dove abbiamo notifiche push server → client, si è optato per Server-Sent Events (SSE), che è una tecnologia più semplice e più adatta al problema.

La differenza fondamentale è che SSE è unidirezionale: solo il server può inviare dati al client, il client non può inviare dati tramite la stessa connessione. Per le notifiche questo va benissimo: le notifiche fluiscono solo dal server al client. WebSocket sarebbe stato necessario eventualmente per la comunicazione bidirezionale, come una chat real-time (anche se parzialmente presente (per la gestione dei ticket, purtroppo l'implementazione avrebbe richiesto tempo ulteriore). Per il solo scopo delle notifiche, SSE è più leggero e più semplice da implementare correttamente.

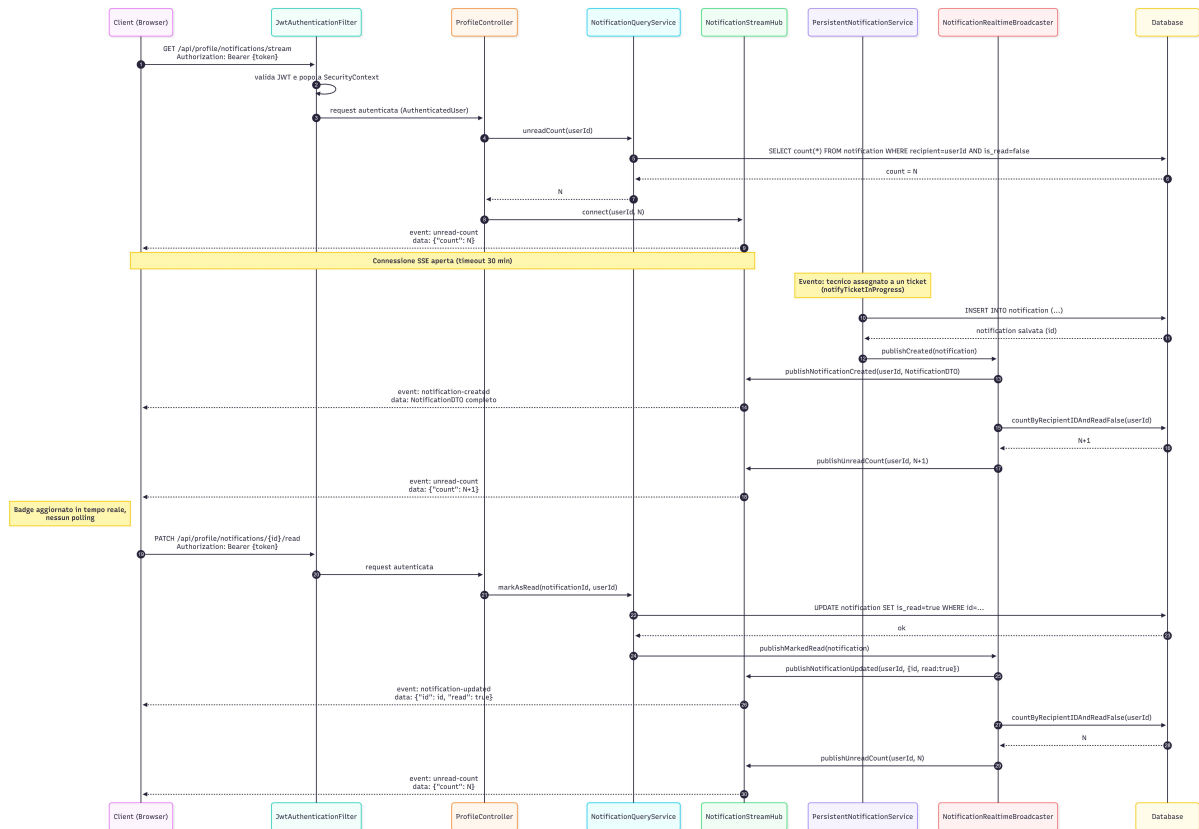
Dal punto di vista pratico, SSE usa una semplice connessione HTTP/1.1 con Content-Type: text/event-stream. Non richiede nessuna libreria aggiuntiva né lato server né lato client, è supportata nativamente da tutti i browser moderni, e funziona senza problemi attraverso proxy e load balancer che a volte creano problemi ai WebSocket.

8.2 Implementazione: NotificationStreamHub e PersistentNotificationService

Come sono strutturate le notifiche? L'architettura delle notifiche è divisa in due componenti principali. **PersistentNotificationService è il punto di ingresso unico per tutte le notifiche:** persiste ogni notifica nel database (in modo che l'utente possa vederle anche se era offline al momento dell'invio), poi chiama il broadcaster SSE per l'invio in tempo reale.

NotificationStreamHub mantiene in memoria una mappa userId → SseEmitter. Cosicché quando un utente si connette all'endpoint /api/profile/notifications/stream, viene creato un SseEmitter associato al suo ID e inserito in questa mappa. Quando arriva una notifica per quell'utente, l'hub cerca l'emitter corrispondente e vi pubblica l'evento.

Ogni emitter ha un timeout di 30 minuti: dopo questo intervallo, la connessione viene chiusa e il client deve riconnettersi. Il client frontend può gestire questa riconnessione in modo trasparente, eventualmente con un exponential backoff.



8.1 — Sequence diagram del flusso notifiche SSE con contatore badge

Il Sequence Diagram mostra come il badge delle notifiche non-lette viene aggiornato in tempo reale senza polling. Quando un tecnico viene assegnato a un ticket, PersistentNotificationService persiste la notifica e pubblica un evento SSE: il client riceve l'evento e aggiorna il badge immediatamente, senza dover interrogare il server periodicamente.

8.3 Sicurezza della Connessione SSE

Un aspetto che abbiamo curato particolarmente è poi la sicurezza della connessione SSE. L'endpoint di stream (/api/profile/notifications/stream) rientra a tutti gli effetti nella catena delle richieste autenticate: in SecurityConfig tutto ciò che sta sotto /api/** richiede un token valido, quindi anche lo stream passa dal JwtAuthenticationFilter come ogni altra chiamata.

Il client Angular apre la connessione passando il token nell'header Authorization come per tutte le altre API protette. Solo dopo che il filtro ha validato il token e popolato il SecurityContext, il controller crea l'SseEmitter e lo associa all'ID utente nella mappa di NotificationStreamHub. In questo modo non c'è bisogno di un meccanismo di autenticazione separato per lo stream: la stessa logica usata per gli endpoint REST copre anche le connessioni SSE.

9. Sicurezza e Autenticazione

9.1 JWT e Autenticazione Stateless

L'autenticazione è basata su JWT (JSON Web Token), che permette un'architettura stateless: il server non mantiene nessuna sessione in memoria. Ogni richiesta porta con sé tutte le informazioni necessarie per autenticare l'utente, codificate nel token.

Il `JwtAuthenticationFilter` intercetta ogni richiesta in arrivo, estrae il token JWT dall'header `Authorization`, lo valida, e popola il `SecurityContext` di Spring con l'utente autenticato. Se il token è assente, malformato o scaduto, la richiesta viene rifiutata con 401 Unauthorized prima ancora di arrivare al controller.

9.2 AccessControlService: Prevenzione IDOR

Anche se il progetto rimane puramente didattico abbiamo voluto sperimentare un ulteriore approccio di sicurezza: oltre all'autenticazione basata su JWT, il sistema implementa controlli di ownership espliciti per prevenire attacchi IDOR (Insecure Direct Object Reference). Questi attacchi si verificano nel momento in cui un utente può accedere a risorse di altri utenti semplicemente cambiando un ID nella richiesta.

`AccessControlService` centralizza questi controlli con metodi chiari:

- **`assertHostOwnsSpace(hostId, spaceId)`**: impedisce che un Host gestisca spazi che non gli appartengono.
- **`assertSelfOrAdmin(requesterId, targetId)`**: permette a un utente di modificare solo il proprio profilo, a meno che non sia SysAdmin.
- **`assertHostOwnsPath(principal, pathHostId, resourceKind)`**: impedisce che un Host operi su URL che riferiscono un altro `hostId` nel path.

Anti-enumeration: quando una risorsa non esiste oppure l'utente non ha i permessi per accedervi, il sistema risponde sempre con 404 NOT_FOUND (non con 403). Questo impedisce a un attaccante di capire se una risorsa esiste semplicemente provando diversi ID.

9.3 SecurityConfig: Regole di Accesso

In SecurityConfig sono definite le regole di accesso a livello di catena di filtri. La scelta che abbiamo fatto è di tenere la configurazione il più semplice possibile: pochi endpoint pubblici (registrazione, login, refresh, Swagger), e tutto il resto sotto /api/** richiede un token JWT valido. Il controllo del ruolo non viene fatto qui in SecurityConfig, ma a livello di service tramite AccessControlService.assertRole(principal, Role.X). Questa scelta tiene la configurazione di Spring Security leggibile e mette i controlli di autorizzazione vicino alla logica di business, dove si capisce subito perché un certo endpoint richiede un determinato ruolo.

```
// Endpoints pubblici (nessuna autenticazione richiesta)
.requestMatchers("/api/auth/register", "/api/auth/register/host", "/api/auth/login",
"/api/auth/refresh").permitAll()
.requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()

// Tutte le altre API richiedono un token valido
.requestMatchers("/api/**").authenticated()

// Tutto il resto è negato
.anyRequest().denyAll()

// Il controllo del ruolo non viene fatto qui: ogni service
// chiama accessControlService.assertRole(principal, Role.WORKER) ecc.
```


10. Gestione degli Errori

La gestione degli errori è stata centralizzata in `RestExceptionHandler`, annotato con `@RestControllerAdvice`. Questo significa che c'è un unico posto in cui tutti gli errori vengono intercettati e trasformati in risposte JSON con struttura uniforme. Non utilizziamo quindi `try/catch` sparsi nei controller.

La risposta di errore ha sempre la stessa struttura:

```
{
  "errorId": "550e8400-e29b-41d4-a716-446655440000",
  "timestamp": "2025-01-15T10:30:00",
  "status": 400,
  "code": "BUSINESS_RULE_VIOLATION",
  "message": "La postazione è già occupata nella fascia oraria selezionata.",
  "path": "/api/workers/bookings",
  "fieldErrors": []
}
```

Il campo `errorId` è un UUID univoco per ogni risposta di errore. Questo è molto utile in produzione: quando un utente segnala un problema, l'`errorId` permette di trovare immediatamente il log corrispondente sul server e riusciamo a ricostruire esattamente cosa è successo.

Eccezione	HTTP	Code JSON	Quando
NotFoundException	404	NOT_FOUND	Risorsa assente o accesso negato (anti-enumeration)
BusinessRuleException	400	BUSINESS_RULE_VIOLATION	Regola dominio violata: slot non disponibile, stato non valido
ForbiddenException	403	FORBIDDEN	Autenticato ma non autorizzato per quell'operazione
UnauthorizedException	401	UNAUTHORIZED	Token mancante, scaduto o invalido
ConflictException	409	CONFLICT	Double-booking, unicità violata

Eccezione	HTTP	Code JSON	Quando
OptimisticLockingFailureException	409	OPTIMISTIC_LOCK	Scrittura concorrente sulla stessa versione
MethodArgumentNotValidException	400	VALIDATION_ERROR	Campi @Valid non rispettati, con dettaglio per campo
Exception (catch-all)	500	INTERNAL_ERROR	Errori non previsti — stack trace completo viene loggato

11. Meccanismi Tecnici Rilevanti

11.1 Lock Pessimistico e Anti-Double-Booking

La prevenzione del double-booking è uno dei problemi più critici in un sistema di prenotazione sotto carico. Immaginate due utenti che premono "Prenota" sul medesimo desk nello stesso momento, potrebbero verificarsi non pochi problemi e questo è un caso d'uso particolarmente realistico e plausibilmente frequente, il sistema deve dunque saperlo gestire correttamente senza dipendere dalla fortuna nel timing.

Come risolviamo? In questo modo: **SmartDesk usa due livelli di protezione. Il primo è il lock pessimistico con SELECT FOR UPDATE: prima di ogni prenotazione, la riga del desk viene bloccata a livello database.** Nessun'altra transazione può leggere la stessa riga in modalità write fino al commit. **Il secondo è la verifica dell'overlap: countDeskOverlapping() e countWorkerOverlapping() eseguono COUNT con condizioni temporali sovrapposte.** Se il conteggio è > 0, la prenotazione viene rifiutata con 409 CONFLICT.

Questo approccio è stato verificato attraverso l'utilizzo di test di concorrenza: BookingConcurrencyIntegrationTest lancia più thread in parallelo che tentano di prenotare lo stesso desk, e verifica che al massimo una prenotazione venga confermata.

11.2 Lock Ottimistico

Per le operazioni meno critiche (aggiornamento profilo, modifica booking) viene usato il lock ottimistico. Le entità User e Booking hanno un campo Long version annotato con @Version. In tal modo JPA include automaticamente una condizione WHERE version = X nelle UPDATE, e lancia ObjectOptimisticLockingFailureException se la versione non corrisponde. Otteniamo quindi che il costo è molto più basso del lock pessimistico (nessun lock a livello DB), ma garantisce comunque che due aggiornamenti concorrenti sullo stesso oggetto non si sovrascrivano silenziosamente, senza accorgercene.

11.3 Generazione Codici Univoci con Retry

I codici prenotazione (6 caratteri alfanumerici) sono generati con CodeUtils.allocateUniqueCode(), mentre i codici ticket (T + 4 caratteri) vengono generati con lo stesso approccio tramite un metodo dedicato nel TicketService. L'alfabeto esclude caratteri ambigui come 0/O e 1/I per evitare confusione nella lettura e trascrizione. Il metodo fa fino a 64 tentativi in caso di collisione prima di lanciare un'eccezione. In questo modo utilizzando un alfabeto di 32 caratteri su 6 posizioni, la probabilità di collisione dopo pochi tentativi è trascurabile, ma il retry garantisce robustezza anche in scenari improbabili.

11.4 Orari di Apertura in JSON

Gli orari settimanali dello spazio (lun-dom, apertura/chiusura, giorno chiuso) sono salvati come colonna TEXT in formato JSON. Questo può sembrare una scelta strana per chi è abituato a modellare tutto con tabelle relazionali, ma ha poi vantaggi concreti: ossia nessuna migrazione del database se si vuole aggiungere supporto per più fasce orarie al giorno, e la lettura è più semplice poiché l'intero oggetto orari è in una sola colonna.

Default intelligente: se openingHoursJson è NULL (spazio appena creato), il sistema usa un default 08:00-20:00 per tutti i giorni. Questo evita quindi che uno spazio appena creato risulti non prenotabile per mancanza di orari configurati.

11.5 Seeder Idempotente

Il dataset demo viene caricato all'avvio dell'applicazione in profilo dev. Il seeder è idempotente: ovvero ogni record viene creato solo se non esiste già nel database. Questo significa che l'applicazione può essere riavviata più volte senza creare duplicati. L'idempotenza viene verificata mediante un test dedicato (DataSeederJsonWorkersTest).

Entità	Quantità	Note
Worker	14	Mix di attivi e bannati per testare la moderazione
Host	6	4 approvati, 2 in pending per testare il flusso di approvazione
Technician	4	Uno per ogni spazio approvato
Spazi	6	4 approvati (Milano, Torino, Bologna, Firenze), 2 in pending
Desk totali	~100	Con varie amenity, alcuni già in manutenzione
Prenotazioni	44	Future, distribuite su 7 giorni
Recensioni	50	Su prenotazioni completate, con risposte host e note admin
Ticket	20	In stati diversi: OPEN, IN_PROGRESS, RESOLVED

12. Strategia di Test

La suite di test copre tutti i livelli dell'applicazione con test unitari, di integrazione e end-to-end. I test usano JUnit 5, Mockito e Spring Boot Test con database H2 in memoria. L'insieme di tutti i test ha **una line coverage superiore all'80%**, garantendo così un'elevata copertura del codice. Per la misurazione della coverage, è stato utilizzato il tool JaCoCo (Java Code Coverage).

Element ^	Class, %	Method, %	Line, %	Branch, %
it.polimi.smartdesk_backend	86% (113/130)	82% (576/696)	85% (2478/2891)	63% (780/1223)
config	37% (6/16)	96% (24/25)	92% (169/183)	72% (65/90)
controller	94% (18/19)	82% (101/123)	81% (220/271)	61% (11/18)
dto	50% (1/2)	33% (1/3)	60% (3/5)	100% (4/4)
event	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
exception	100% (6/6)	89% (25/28)	85% (77/90)	68% (11/16)
mapper	60% (3/5)	87% (14/16)	97% (102/105)	77% (42/54)
model	88% (8/9)	76% (20/26)	75% (48/64)	65% (31/47)
repository	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
security.filter	100% (1/1)	100% (3/3)	81% (18/22)	70% (7/10)
service	98% (54/55)	83% (315/378)	85% (1597/1877)	62% (572/922)
support	100% (4/4)	87% (7/8)	80% (29/36)	53% (15/28)
util	100% (12/12)	77% (66/85)	90% (215/237)	64% (22/34)
SmartdeskBackendApplication	0% (0/1)	0% (0/1)	0% (0/1)	100% (0/0)

12.1 Test Unitari: Controller

Ogni controller ha una classe di test dedicata (ad esempio WorkerBookingControllerTest, HostTicketControllerTest) che usa @WebMvcTest per testare la validazione degli input, l'autorizzazione e le risposte HTTP senza avviare l'intero application context. I service vengono mockati con Mockito, in modo che i test del controller si concentrino solo sul comportamento del livello HTTP.

Questi test coprono grosso modo tutti gli scenari come: richiesta con body non valido (400 con fieldErrors), richiesta con token mancante (401), richiesta con ruolo sbagliato (403), e i casi di successo standard (200/201/204).

12.2 Test Unitari: Service e Model

I service critici sono testati con unit test puri, senza Spring context: AuditLogServiceTest, SysAdminServiceTest, DeskAvailabilityServiceTest, DeskServiceTest, DeskStateMachineTest, TicketStateMachineTest. ModelBehaviorTest verifica i vincoli di dominio sulle entità. Questi test sono pertanto i più rapidi da eseguire ed anche i più specifici: testano una singola unità di codice con tutti i collaboratori mockati.

12.3 Test di Integrazione e Concorrenza

BackendEndToEndFlowTest simula invece il flusso completo prenotazione → cancellazione → notifica waitlist su database H2 reale. Questo test è più lento ma verifica che tutti i layer funzionino correttamente insieme.

Poi troviamo anche **BookingConcurrencyIntegrationTest** che è probabilmente il test più interessante: lancia più thread in parallelo che tentano di prenotare lo stesso desk nella stessa fascia oraria, e verifica che esattamente una prenotazione venga confermata e tutte le altre vengano rifiutate con 409 CONFLICT. Questo test garantisce che il lock pessimistico funzioni correttamente.

12.4 Test di Sicurezza

JwtAuthenticationFilterTest verifica i casi limite del filtro JWT: token mancante, token malformato, token scaduto, token con firma errata. **RestExceptionHandlerTest** verifica la struttura delle risposte di errore per ogni tipo di eccezione, garantendo che il contratto dell'API sia sempre rispettato.

13. Architettura Frontend

Il frontend di SmartDesk è stato realizzato come applicazione Angular e rappresenta il punto di accesso unico per tutti i ruoli del sistema. La parte client non contiene le regole di dominio più delicate, che restano nel backend, ma organizza navigazione, sessione, autorizzazioni lato client, comunicazione con le API e aggiornamento dell'interfaccia.

Questa separazione ci ha permesso di mantenere una struttura abbastanza semplice da seguire anche durante lo sviluppo. Ogni pagina mostra i dati necessari al ruolo corrente, mentre servizi, guard, interceptor e modelli TypeScript evitano di ripetere la stessa logica in più componenti.

13.1 Stack Tecnologico

Il client usa Angular 21 con TypeScript, RxJS e SCSS. Angular è stato scelto perché fornisce routing, Dependency Injection e una struttura chiara per dividere l'applicazione in parti indipendenti. TypeScript è importante perché permette di modellare utenti, desk, prenotazioni, ticket e notifiche con tipi controllati, riducendo gli errori quando i dati arrivano dalle API.

Per l'interfaccia sono stati usati Bootstrap 5, MDB Angular UI Kit e componenti sviluppati nel progetto. La parte grafica non è quindi lasciata alle singole pagine, ma passa da stili comuni, variabili SCSS e componenti condivisi. I test frontend sono eseguiti con Vitest e si concentrano sui punti in cui un errore avrebbe conseguenze visibili sull'applicazione.

13.2 Organizzazione dell'applicazione

Il codice è diviso in tre aree principali. La cartella **core** contiene servizi, modelli, guard, interceptor e utility usate da tutta l'applicazione. La cartella **features** raccoglie le pagine dei ruoli worker, host, technician e sys admin. La cartella **shared** contiene gli elementi riutilizzabili, come navbar, menu, badge di stato, modali, toast, calendario di ricerca e pannello chat dei ticket.

Il componente **AppComponent** resta il contenitore stabile dell'applicazione e ospita il **router outlet**. Le rotte pubbliche gestiscono landing page, login e registrazione, mentre le aree riservate sono caricate separatamente in base al ruolo. In questo modo il layout generale rimane coerente, ma il contenuto centrale cambia senza ricaricare tutta la pagina.

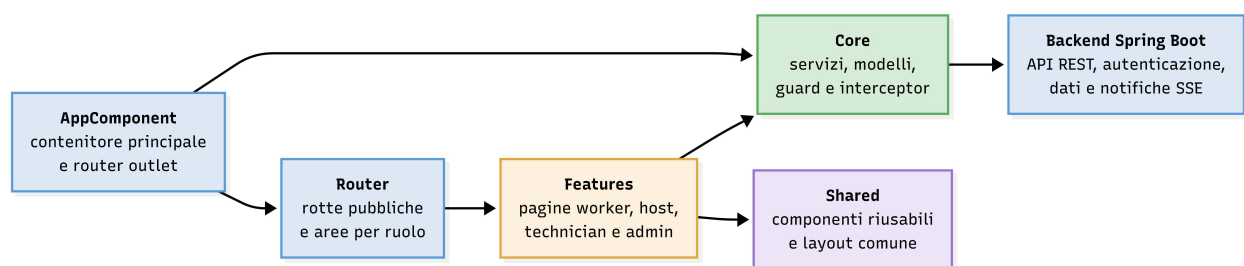


Figura 13.1: Schema di organizzazione del frontend Angular

13.3 Routing, shell e navigazione per ruolo

La navigazione è stata costruita partendo dai quattro ruoli del sistema. Dopo il login, l'utente viene indirizzato verso la propria area e vede solo le sezioni compatibili con il profilo caricato. Questa scelta rende più leggibile il progetto, perché ogni ruolo ha file di routing e pagine dedicate, ma al tempo stesso evita di duplicare il layout generale.

Per le pagine riservate viene usato **RoleShellComponent**. Questo componente contiene la struttura comune della dashboard e ricava il menu da una configurazione centrale. Il risultato è che worker, host, technician e sys admin condividono la stessa base visiva, ma hanno voci di menu diverse e contenuti specifici.

Le rotte sono protette da **authGuard** e **roleGuard**. Il primo controlla che esista una sessione valida, mentre il secondo verifica che il ruolo presente nel profilo e nel token sia coerente con la pagina richiesta. Questo controllo non sostituisce quello del backend, però evita di mostrare schermate non utilizzabili e migliora il comportamento dell'interfaccia.

13.4 Comunicazione con il backend

La comunicazione con Spring Boot avviene tramite servizi Angular basati su **HttpClient**. Ogni servizio segue una macro area funzionale, come autenticazione, prenotazioni, ticket, recensioni,

gestione host, gestione admin e notifiche. In questo modo i componenti non costruiscono direttamente le richieste HTTP, ma chiamano metodi più leggibili e vicini al significato della pagina.

Le richieste passano dagli interceptor prima di arrivare al backend. JwtInterceptor aggiunge il token alle chiamate protette, SessionExpiryInterceptor gestisce i casi di sessione scaduta e AuthRefreshInterceptor prova a rinnovare il token quando riceve una risposta 401. Nel codice è presente anche una protezione contro più refresh paralleli, utile quando diverse chiamate scadono nello stesso momento.

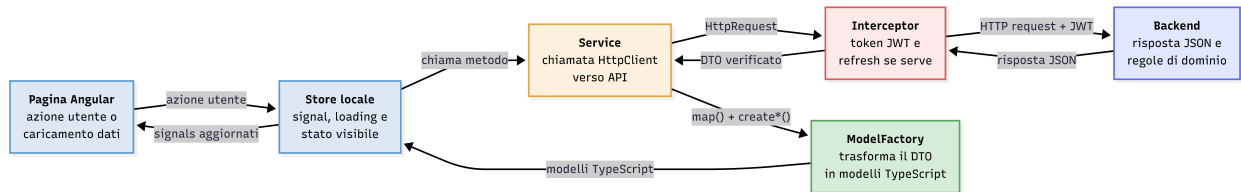


Figura 13.2: Flusso di richiesta tra pagina, servizi e backend

13.5 Stato delle pagine e gestione dei dati

Per lo stato locale delle pagine sono stati usati soprattutto i signal di Angular. I signal conservano dati caricati dal backend, stati di caricamento, errori e valori derivati usati dai template. Questo approccio rende chiaro quali informazioni sono direttamente salvate nella pagina e quali invece vengono calcolate a partire da altre proprietà.

Quando una schermata diventava troppo complessa, una parte della logica è stata tenuta vicino al componente che la utilizza tramite signal e computed locali. Non ci sono store globali dell'intera applicazione: lo stato condiviso tra login, navbar e guard passa da AuthStateService, mentre per il resto ogni pagina mantiene da sé i propri signal di caricamento, errori e dati derivati.

Questa soluzione è utile soprattutto nella ricerca delle postazioni. La pagina deve tenere insieme città, spazio selezionato, giorno, orario, desk disponibili, prenotazioni già presenti e lista d'attesa. Separare lo stato dalla vista rende il codice meno confuso e permette al componente di concentrarsi su ciò che deve mostrare all'utente.

13.6 Modelli TypeScript e programmazione orientata agli oggetti

Nel frontend non sono state usate soltanto interfacce semplici per rappresentare le risposte JSON. In diversi punti abbiamo preferito classi TypeScript vere e proprie, così da portare nel client una parte del modello applicativo già presente lato backend. La gerarchia degli utenti parte da AbstractUser e viene specializzata in Worker, Host, Technician e SysAdmin.

Le sottoclassi permettono di ottenere comportamenti diversi mantenendo un riferimento comune. Un componente può lavorare con un utente generico e usare metodi come `getRole`, `getDashboardTitle` e `canOpenTicket` senza conoscere subito il tipo concreto. Questa scelta riduce codice ripetuto e rende più ordinata la gestione delle differenze tra ruoli.

Lo stesso principio viene usato per i desk. L'interfaccia `IDeskState` definisce un contratto comune, mentre classi come `AvailableDeskState`, `MaintenanceDeskState`, `PendingInspectionDeskState` e `DecommissionedDeskState` rappresentano stati diversi della postazione. La `ModelFactory` completa questo lavoro trasformando le risposte del backend in oggetti TypeScript più comodi da usare nella UI.

13.7 Flussi principali dell'interfaccia

Il flusso più importante per il worker è quello di prenotazione. L'utente sceglie città, spazio, data e fascia oraria, poi il frontend richiede al backend le postazioni compatibili. La risposta viene convertita nei modelli del client e mostrata nella pagina di ricerca. Quando la prenotazione viene confermata, `BookingService` invia la richiesta e la schermata aggiorna anche notifiche locali e stato visibile.

Per l'host il frontend raggruppa attività amministrative più varie. Le pagine permettono di gestire spazi, stanze, postazioni, tecnici, segnalazioni e recensioni. La parte più delicata è mantenere coerente ciò che si vede con ciò che succede nel backend, per esempio quando una postazione viene chiusa, rimessa disponibile o collegata a un ticket.

Per il technician il centro dell'interfaccia è la gestione dei ticket. Le schermate distinguono tra segnalazioni in attesa, ticket assegnati e manutenzione delle postazioni. L'obiettivo della UI è mostrare vicino all'azione anche il contesto necessario, quindi spazio, desk, priorità, stato e storico della conversazione.

L'area sys admin è invece orientata al controllo della piattaforma. Da qui vengono gestiti utenti, richieste di approvazione, spazi, prenotazioni, recensioni e log. Anche in questo caso il frontend non decide le regole finali, ma organizza le operazioni in pagine separate e manda al backend le richieste di validazione o modifica.

13.8 Sessione, notifiche e aggiornamenti della UI

La sessione dell'utente è mantenuta in `AuthStateService`, che conserva access token, refresh token e dati del profilo. Al refresh della pagina il servizio prova a ricostruire l'utente salvato e a ripristinare lo stato di autenticazione. Usare un servizio dedicato evita che login, logout, guard, interceptor e navbar gestiscano ognuno la sessione in modo diverso.

Le notifiche usano lo stesso modello SSE descritto nella parte backend. `NotificationService` apre lo stream verso l'endpoint del profilo e interpreta eventi come nuova notifica, cambio dello stato di lettura e azzeramento del contatore. Il badge nella navbar viene aggiornato senza polling continuo, mentre le

pagine possono comunque richiedere un refresh esplicito quando una prenotazione, un ticket o una recensione generano nuovi eventi.

13.9 Interfaccia, componenti condivisi e stile

Dal punto di vista grafico abbiamo cercato di mantenere la stessa tipologia di vista tra ruoli diversi. Navbar, menu di ruolo, badge, calendario di ricerca, modali di conferma, toast e pannello chat dei ticket sono stati estratti come componenti condivisi. Questo evita di correggere più volte lo stesso comportamento e rende più uniforme l'esperienza utente.

Gli stili sono organizzati in un file SCSS globale (styles.scss) e in file SCSS dedicati ai singoli componenti, con variabili condivise per colori, spaziature e tipografia. Questa struttura riduce le regole sparse dentro le singole pagine e mantiene una resa coerente in dashboard, form, liste e viste di dettaglio.

14. Considerazioni Finali

14.1 Punti di Forza da evidenziare

Guardando al progetto nel suo complesso, ci sono alcune scelte che siamo contenti di aver intrapreso fin da subito e che hanno reso lo sviluppo molto più ordinato facilitandoci lo sviluppo.

La scelta chiaramente di un'architettura MVC e quindi la conseguente separazione netta delle responsabilità, controller sottili, service ricchi, repository specifici per dominio, ha reso possibile testare ogni parte in isolamento e ha ridotto significativamente il numero di bug introdotti durante le modifiche. Quando si modifica `BookingCreationService`, non c'è nessun rischio di rompere accidentalmente la logica dei ticket.

Il pattern State per desk e ticket si è rivelato particolarmente utile man mano che aggiungevano nuove transizioni. Ogni volta che si è dovuta aggiungere una nuova operazione su un desk, era chiaro esattamente dove inserirla (nella classe dello stato corretto) e quali stati dovevano lanciarla come eccezione (tutti gli altri). Senza questo pattern, molto probabilmente la logica si sarebbe accumulata in un unico metodo con un lungo switch sullo stato, preferibilmente evitabile.

E poi la scelta delle notifiche via SSE possiamo dire che funziona abbastanza affidabilmente, con il pattern Observer/AFTER_COMMIT che garantisce che le notifiche vengano inviate solo dopo il commit della transazione. In questo modo non abbiamo mai riscontrato casi di notifiche fantasma per prenotazioni poi rollbackate.

Abbiamo inoltre cercato di rendere il frontend quanto più coerente con l'architettura del backend. I ruoli sono separati, le chiamate HTTP sono centralizzate nei servizi, la sicurezza lato client è gestita da guard e interceptor e lo stato delle pagine è mantenuto vicino ai componenti che lo utilizzano.

La scelta di Angular e TypeScript si è rivelata utile per l'ordine e l'approccio più lato business ma avremmo potuto utilizzare anche React. Modelli, componenti condivisi e notifiche in tempo reale permettono all'applicazione di rimanere estendibile senza perdere leggibilità.

14.2 Trade-off e Limitazioni Note

Ogni sistema ha le sue limitazioni, incluso ovviamente **SmartDesk** che **rimane un software puramente didattico**. Alcune di queste sono ad esempio:

- **Sicuramente in primis aggiungere una mappa degli uffici** con i desk selezionabili cliccando sulla mappa visualizzata.
- **Nessuna paginazione sulle liste più grandi** (prenotazioni admin, ticket host): da aggiungere con Spring Data Pageable per evitare scroll infinito.
- **Technician legge ticket in tutte le stanze dei propri spazi senza filtro per specializzazione**: semplificazione consapevole poiché a livello didattico.

- **Supporto al pagamento integrato** (es. Stripe) per prenotazioni a pagamento.
- **Aggiunta notifiche email/push** oltre alle notifiche in-app SSE.
- **Dashboard statistiche host**: occupazione desk per fascia oraria, trend settimanali, revenue per spazio.